COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

The C-language bridge

Actual C compilation and C++ integration

Four exercises: C17 compilation, resource ownership, structure layout and status-return failures.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

Why this is part of Volume 03 3

C01 — Compile and call actual C 3

C02 — Match ownership across the boundary 4

C03 — Separate native layout from wire layout 5

C04 — Status returns, failure injection and output atomicity 6

Run and review 6

Continuity with the original program 7

Primary reference directory 7

Reading by topic 9

C-language bridge for gaming hardware

Why this is part of Volume 03

The original hardware/firmware route requires a C bridge after Module15. This bundle supplies four exercises and four complete reference programs. C and C++ are distinct languages; a `.c` filename processed by a C++ compiler is not evidence of actual C-language compatibility. The build declares both languages, requires C17 for the C sources and C++20 for the C++ sources, and rejects compiling the C implementation as C++. [R11; P4-05]

The mock device performs no hardware I/O and needs no board or permissions. Its allocation-failure and live-count hooks are deliberately single-threaded teaching instrumentation. Do not treat the API as a driver, thread-safe library or portable binary interface across arbitrary compilers. It demonstrates a same-toolchain C/C++ boundary that you can inspect and test.

C01 — Compile and call actual C

Learn the boundary

The C header includes C headers such as `<stddef.h>` and `<stdint.h>` and declares functions and plain C-compatible types. The conditional `extern "C"` block changes language linkage when the header is seen from C++; a C compiler never sees those C++ keywords. It does not translate C++ classes, guarantee cross-compiler ABI compatibility or make an unsafe function safe.

In C, `struct ge_device` names a structure type directly; C++ permits more contexts where the struct tag can be used without the keyword. Do not introduce templates, references, exceptions, overloaded declarations or standard-library containers into the public C interface. Pass explicit lengths with pointer-based buffers. Document what each pointer owns, borrows or writes.

The function `ge_apply_damage(hp, damage, out_hp)` accepts nonnegative hp and damage at most1,000,000. It produces max(0,hp-damage) and returns a status code. A null output or invalid domain is an error. Failed calls leave a valid caller-provided output unchanged. Avoid an arithmetic expression whose safety is justified only after it has already executed.

Work and evidence

Inspect `code/c_bridge/ge_c_api.h`, `ge_c_api.c` and `c01_basics.c`. Capture the verbose compiler command and the reported `__STDC_VERSION__`; the source requires at least the C17 value201710L. Then implement your own small C function that clamps a nonnegative cooldown under a documented maximum, call it from a C test and from a C++ test, and retain identical status semantics.

Pass when the C implementation is compiled as C, both callers pass, a malformed/null case is rejected without modifying valid output storage, and you can explain why `extern "C"` belongs in the header's C++ branch. Add your files to an explicit CMake target; do not include a `.c` implementation into a C++ source as a substitute.

C01 quiz

1. Does `extern "C"` make a C++ class layout usable by every C compiler?
2. What evidence distinguishes compiling as C from merely using a `.c` filename?
3. Why should a failed status return not leave the output's meaning undocumented?

C02 — Match ownership across the boundary

Learn the ownership protocol

An opaque handle hides representation from callers, not lifetime responsibility. `ge_device_create(id, &handle)` returns a successful uniquely owned handle or an error. A nonnull output pointer is required; failed creation sets the output handle to null. Only `ge_device_destroy` releases a successful handle. Destroying null is allowed; destroying the same live handle twice is not.

C callers commonly initialize a handle to null and use one cleanup path that checks or safely releases it. C++ callers can immediately adopt it into `std::unique_ptr<ge_device, Deleter>`, where the deleter calls the C destroy function. Default `delete` is not the matching operation for memory obtained by this C implementation. The opaque structure need not be complete in the C++ caller when a suitable custom deleter is used.

Ownership crosses an API boundary only under the stated protocol. A borrowed pointer returned for observation must not be freed by its observer. A C++ exception must not unwind through a C calling convention boundary as an undocumented error mechanism; translate internal exceptions to declared status results at an appropriate wrapper boundary. [R11: R and I]

Work and evidence

Study `c02_ownership.cpp`. Implement a C++ wrapper with no copy ownership, move support, an explicit empty state and the correct deleter. Show a successful create, a move, a scope exit and a failure after acquisition. Use `ge_live_devices()` to verify zero live devices after each isolated test. Explain why a live counter is instrumentation, not the allocator itself.

Pass when each successful allocation is released exactly once, the source loses ownership on wrapper move, failure paths are covered, and no caller uses `delete`, `free` or a second destroy on a borrowed or already-released handle. Document the single-threaded scope of the mock failure hook.

C02 quiz

1. Who releases a handle returned by a successful create call?
2. Why is a custom unique_ptr deleter needed here?
3. Does a pointer being opaque prevent a double release?

C03 — Separate native layout from wire layout

Learn what an observation means

Native structure layout is affected by member types, alignment and padding under the implementation's ABI. `sizeof`, C `_Alignof` and `offsetof` let you inspect those properties. Printing the offsets on one machine is an observation about that build, not a specification for every platform.

The bridge packet has fields version:uint8, id:uint32, hp:uint16 and flags:uint8. Its **wire format is exactly eight bytes**: one version byte, four little-endian ID bytes, two little-endian HP bytes and one flags byte. The native struct may occupy more than eight bytes. Serialization deliberately writes fields, never the entire native object representation.

For version1, ID0x12345678, HP0x1234 and flags5, the golden bytes are:

```
01 78 56 34 12 34 12 05
```

The decoder validates exact length, version and pointers before committing output. Native pointer fields, C++ vector internals and padding bytes are not portable payloads. A round trip alone is insufficient evidence: an encoder and decoder can share an endianness bug and still agree. Retain a separately specified golden byte sequence.

Work and evidence

Run `c03_layout.c` and record actual size, alignment and member offsets. Reorder a separate experimental native struct and observe any differences without changing the wire specification. Implement an additional field-by-field packet version in your own code, define every offset and byte order, and reject unsupported versions explicitly.

Pass when native observations are labeled platform-specific, the original eight-byte golden format is exact, round trips preserve fields, and truncation/version errors leave the documented destination unchanged. Do not “fix” a wire mismatch by globally packing every native structure or by assuming that packing solves byte order.

C03 quiz

1. Why can sizeof(ge_packet) exceed the eight-byte wire length?
2. Why does encode-then-decode equality not establish the correct byte order?
3. What should happen when the packet version is unsupported?

C04 — Status returns, failure injection and output atomicity

Learn the error protocol

The mock interface distinguishes success, invalid input, range failure and allocation failure. `ge_device_create` has a null-on-failure ownership output policy, whereas the numeric/packet routines preserve the previous caller output on failure. Those are different useful contracts, not interchangeable conventions. Read the function-specific documentation.

Failure injection forces the next eligible device allocation to fail in a deterministic test. It allows a path that is difficult to trigger reliably using real out-of-memory conditions. The hook does not simulate every possible allocator or operating-system failure, and its global single-threaded state is not a concurrency model.

For an all-or-nothing decoder, validate the input and assemble a local candidate, then assign the output after every check succeeds. Tests initialize output with recognizable sentinel values, trigger one failure and compare the complete prior state. Do not use an uninitialized sentinel or read invalid storage to test a failed call.

Work and evidence

Study `c04_errors.cpp`. Add independent tests for null output, null input, unsupported version, short and long packet length, invalid numeric domain and forced allocation failure. Verify the correct status and the correct unchanged/null policy on each path. After a failed create, call create again and show normal behavior resumes under the mock hook's contract.

Pass when no invalid call silently succeeds, no failure is mistaken for a usable handle, outputs follow their documented policy and all successful handles are released. Show how your C++ wrapper translates status into an optional/result/exception at its own boundary without allowing an exception to cross the declared C interface.

C04 quiz

1. Why are forced allocation-failure tests more reproducible than trying to exhaust all machine memory?
2. Are unchanged-output and null-on-failure-output policies the same contract?
3. What extra work is required before exposing this mock API concurrently?

Run and review

From the package root after a normal build:

```
ctest --test-dir build/debug -C Debug -L c_bridge -V
cmake --build build/debug --config Debug --verbose
```

The verbose command may show no compiler invocation when nothing needs rebuilding. Use a fresh build directory or change one source and rebuild to capture the actual compilation stage. The four bridge reference programs should pass, but your independently written variants and boundary tests are the bridge submission.

Keep the C-language check, C++ wrapper ownership map, layout observations, exact byte fixtures, failure table and toolchain fingerprint in Gate C. Return to the chosen compiler's C documentation and the later RTOS/platform examples before real hardware work. This bridge intentionally does not teach register access, interrupts, volatile semantics, DMA or synchronization in isolation from the later systems and hardware modules.

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.